

CHAPTER – 05

LIST

Introduction to Lists in Python

A **list** is a built-in data type in Python used to **store multiple items** in a **single variable**. It can store elements of different data types such as integers, floats, strings, etc

Key Features of Lists:

1. **Ordered:**
Items in a list maintain their order. When you add a new item, it is placed at the end.
2. **Changeable (Mutable):**
You can change, add, or remove items after the list has been created.
3. **Allows Duplicates:**
Lists can have duplicate values.
4. **Indexed:**
Just like strings, list items can be accessed using index numbers (starting from 0).

Syntax of List :

```
Code : my_list = ["a", "b", "c"]  
  
print(my_list)
```

Mixed Data Types in a List :

```
code : mixed_list = [10, 3.14, "Hello", True]  
  
print(mixed_list)
```

List Indexing in Python

Just like strings, **lists are indexed** — each element has a position number, starting from 0.

Indexing Syntax:

```
code : list_name[index]
```

example index : fruits = ["apple", "banana", "cherry", "mango"]

Element	Index	Negative Index
"apple"	0	-4
"banana"	1	-3
"cherry"	2	-2
"mango"	3	-1

Accessing Elements:

```
print(fruits[0]) # apple
print(fruits[2]) # cherry
print(fruits[-1]) # mango (last element)
print(fruits[-3]) # banana
```

Index Error Example:

```
print(fruits[4]) # IndexError: list index out of range
```

LIST METHOD :

consider the following list :

Method	Description	Example Code
append(x)	Adds an element x at the end of the list	fruits.append("grape")
clear()	Removes all elements from the list	fruits.clear()
copy()	Returns a shallow copy of the list	new_list = fruits.copy()
count(x)	Returns the number of occurrences of value x in the list	fruits.count("apple")

index(x)	Returns the index of the first occurrence of value x	fruits.index("banana")
insert(i, x)	Inserts element x at position i	fruits.insert(1, "orange")
pop(i)	Removes and returns the item at index i (last item by default)	fruits.pop() or fruits.pop(2)
remove(x)	Removes the first occurrence of value x from the list	fruits.remove("apple")
reverse()	Reverses the order of the list	fruits.reverse()
sort()	Sorts the list in ascending order (or descending with reverse=True)	fruits.sort() or fruits.sort(reverse=True)
len(list)	Returns the number of items in the list	len(fruits)
type(obj)	Returns the type of the object	type(fruits)
Slicing	Used to access a part of the list using [start:end:step]	fruits[1:3] or fruits[::-1]

Code :

```
fruits = ["apple", "banana", "cherry"]
```

append

```
fruits.append("mango")    # ['apple', 'banana', 'cherry', 'mango']
```

insert

```
fruits.insert(1, "orange")  # ['apple', 'orange', 'banana', 'cherry', 'mango']
```

remove

```
fruits.remove("banana")    # ['apple', 'orange', 'cherry', 'mango']
```

pop

```
fruits.pop()          # removes 'mango'
```

reverse

```
fruits.reverse()      # ['cherry', 'orange', 'apple']
```

sort

```
fruits.sort()         # ['apple', 'cherry', 'orange']
```

count

```
print(fruits.count("apple")) # 1
```

index

```
print(fruits.index("orange")) # 2
```

copy

```
new_list = fruits.copy()
```

clear

```
fruits.clear()        # []
```

Creating Lists in Python

In Python, a list is a collection which can store multiple items. Lists can be created in the following 3 ways:

(i) Homogeneous List : All elements in the list are of the same data type (e.g., all integers).

```
numbers = [1, 2, 3, 4, 5, 6, 7]
```

```
print(numbers)
```

(ii) Heterogeneous List

List can contain **different data types** — integers, floats, strings, booleans, etc.

```
mixed = [1, "a", 2.5, True]
```

```
print(mixed)
```

(iii) Nested List

A list inside another list (multi-dimensional list).

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
print(matrix)
```

Accessing List Items

You can access individual items in a list using their index number indexing start from 0:

Example 1 – Simple List:

```
numbers = [1, 2, 3, 4, 5, 6]

print(numbers[1]) # Output: 2 (second element)
```

Example 2 – Nested List (2D List):

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(matrix[1][2]) # Output: 6 (3rd item in 2nd list)
```

3. Modifying List Elements

Lists in Python are **mutable**, meaning you can **change** their values.

Example:

```
numbers = [1, 2, 3, 4, 5, 6]

numbers[3] = 55

print(numbers)
```

Output:[1, 2, 3, 55, 5, 6]

4. Adding Elements to a List

You can add elements using **3 main methods**:

Method	Description	Example
append(x)	Adds x at the end of the list	numbers.append(10)
insert(i, x)	Inserts x at position i	numbers.insert(2, 99)
extend(list2)	Adds all items from list2 to the list	numbers.extend([7, 8, 9])

Code Example:

```
numbers = [1, 2, 3]

numbers.append(4)    # [1, 2, 3, 4]

numbers.insert(1, 99) # [1, 99, 2, 3, 4]

numbers.extend([5, 6]) # [1, 99, 2, 3, 4, 5, 6]

print(numbers)
```

5. Removing Elements from a List in Python

You can remove elements from a list using several built-in methods depending on what you want to do.

Method	Use Case	Example
<code>remove(x)</code>	Remove by value	<code>list.remove("apple")</code>
<code>pop(i)</code>	Remove by index , or last if not given	<code>list.pop(2)</code> or <code>list.pop()</code>
<code>clear()</code>	Remove all elements	<code>list.clear()</code>
<code>del list[i]</code>	Delete element at index i	<code>del list[0]</code>
<code>del list</code>	Delete the whole list	<code>del list</code>

PRACTICE SET

Python Practice Questions — List Operations

Instructions:

Write a Python program to perform the following list operations.

Question 1:

Create a list of at least 10 elements.

Access and print the **3rd and 4th** position elements from the list.

Question 2:

Modify the elements at the **2nd and 9th** positions of the list.

Print the list after modification.

Question 3:

Add a new element at the **end** of the list using a suitable method.

Also, **insert** an element at a **specific position** (e.g., 5th position).

Question 4:

Remove an element from the list at a **specified index** (e.g., 6th position).

Print the list after removal.

Question 5:

Slice the list into multiple parts (e.g., part1 = first 4 elements, part2 = next 4).

Access and print **any two parts** of the list.

Question 6:

Sort the list in both:

- **Ascending order**
- **Descending order**

Print both results.